

Introduction to Deep Learning for Natural Language Processing

7. BERT

ELMo (복습)

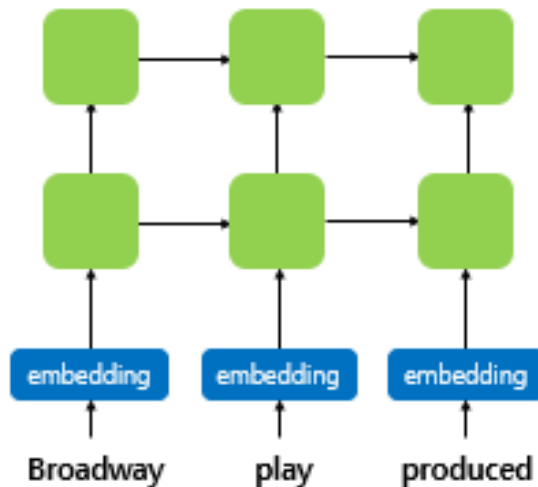
엘모(ELMo) – Embeddings from Language Model

- 엘모는 직역하면 ‘언어 모델로부터 얻는 임베딩’
- 엘모는 BiLM(양방향 언어 모델)을 사용한다.
- 엘모는 순방향과 역방향으로 RNN 언어 모델을 따로 학습시킨다.

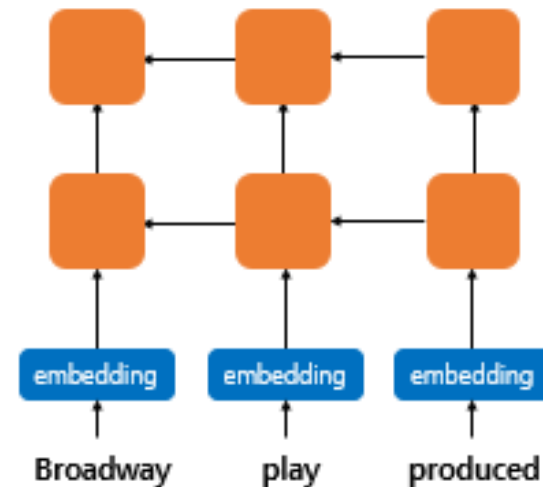


이 캐릭터 이름이 엘모.

순방향 언어 모델
(Forward Language Model)

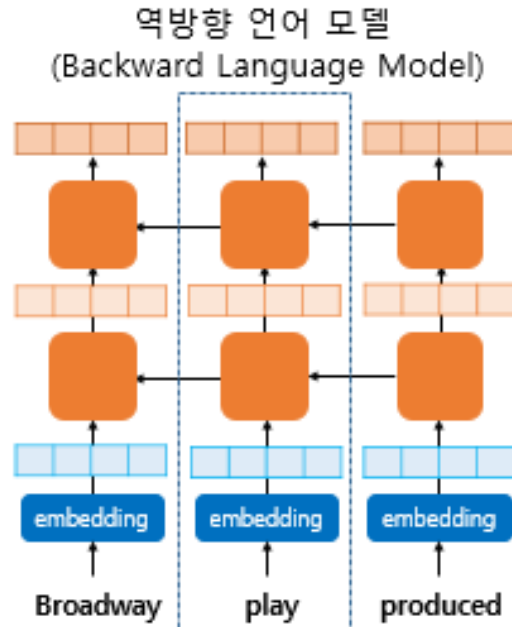
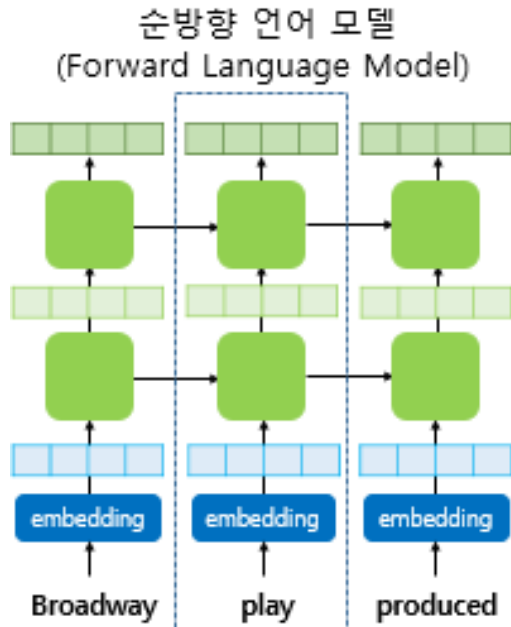


역방향 언어 모델
(Backward Language Model)



엘모(ELMo) – Embeddings from Language Model

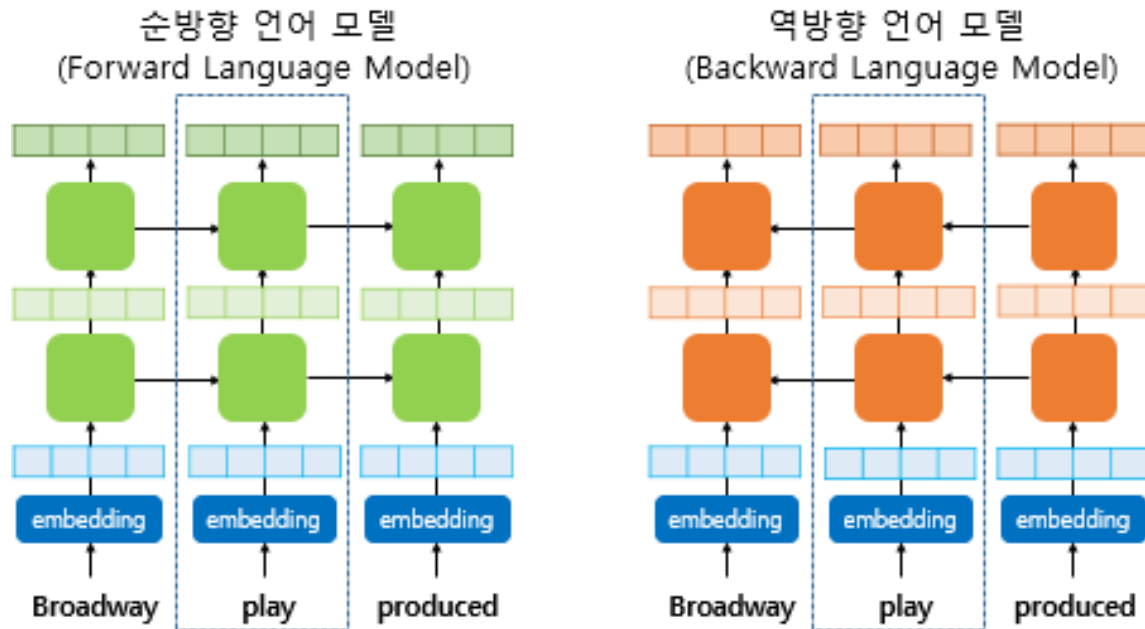
- 엘모는 BiLM(양방향 언어 모델)을 사용한다.
- 엘모는 순방향과 역방향으로 언어 모델을 따로 학습시킨다. 이것이 사전 학습(pre-training) 단계이다.
- 예를 들어 단어 'play'에 대한 임베딩을 얻는다고 가정해보자.



ELMo는 좌측 점선의 사각형 내부의
각 층의 결과값을 재료로 사용한다.

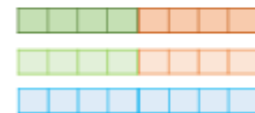
엘모(ELMo) – Embeddings from Language Model

- 예를 들어 단어 'play'에 대한 임베딩을 얻는다고 가정해보자.

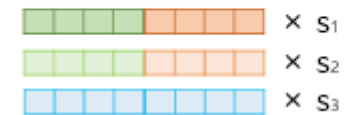


ELMo는 좌측 점선의 사각형 내부의
각 층의 결과값을 재료로 사용한다.

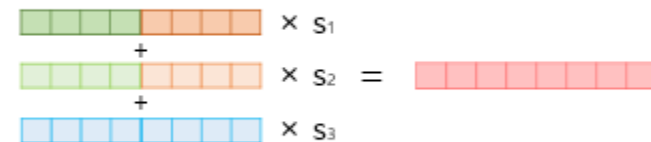
- 1) 각 층의 출력값을 연결(concatenate)한다.



- 2) 각 층의 출력값 별로 가중치를 준다.



- 3) 각 층의 출력값을 모두 더한다.



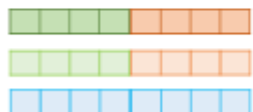
2)번과 3)번의 단계를 요약하여 가중합(Weighted Sum)을 한다고 할 수 있습니다.

- 4) 벡터의 크기를 결정하는 스칼라 매개변수를 곱한다.

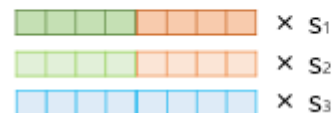
$$\gamma \times \text{red_bar} = \text{red_bar}$$

엘모(ELMo) – Embeddings from Language Model

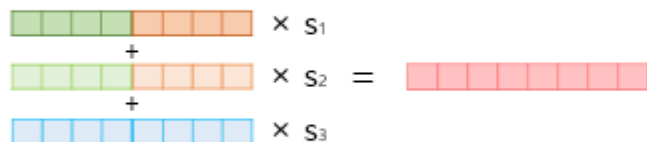
1) 각 층의 출력값을 연결(concatenate)한다.



2) 각 층의 출력값 별로 가중치를 준다.



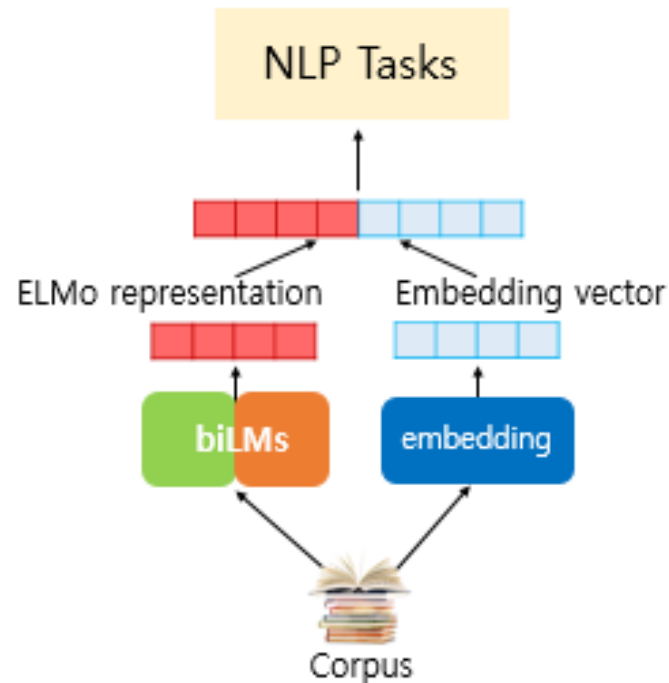
3) 각 층의 출력값을 모두 더한다.



2)번과 3)번의 단계를 요약하여 가중합(Weighted Sum)을 한다고 할 수 있습니다.

4) 벡터의 크기를 결정하는 스칼라 매개변수를 곱한다.

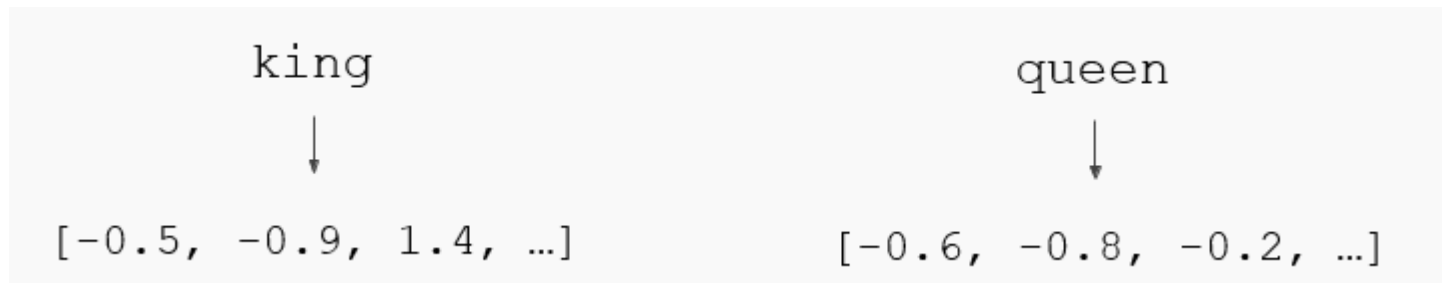
$$\gamma \times \text{[red vector]} = \text{[red vector]}$$



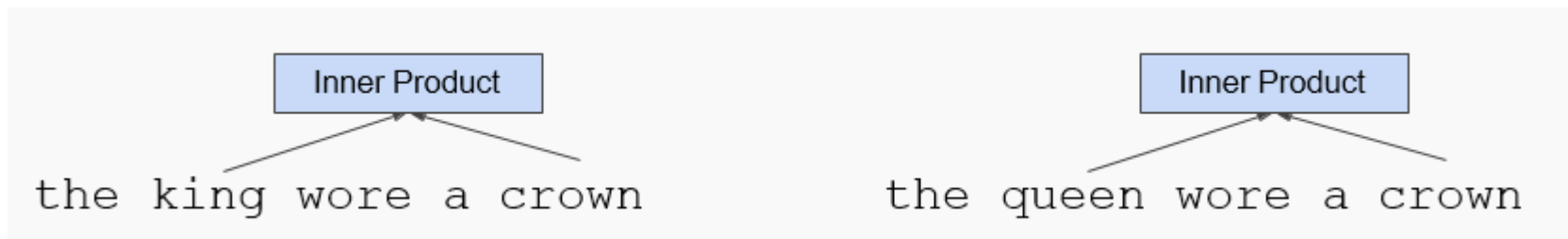
Pre-training in NLP

Pre-training in NLP

- 워드 임베딩은 딥 러닝 자연어 처리의 기본.



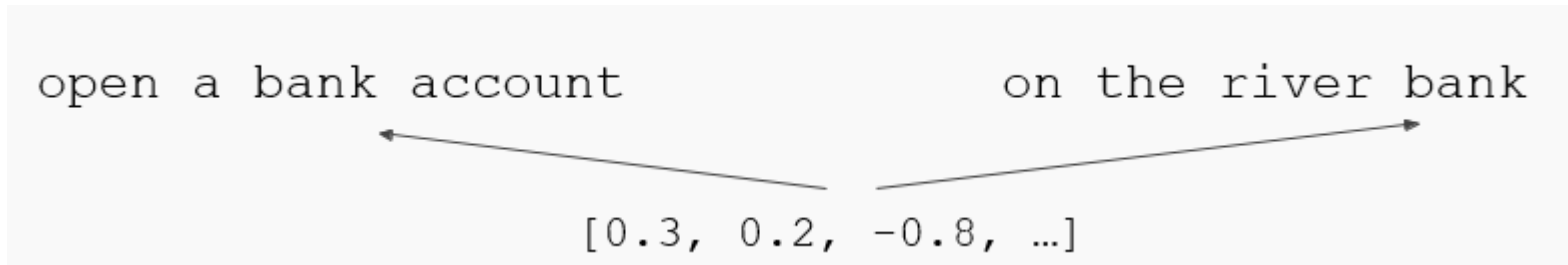
- 사전 훈련된 임베딩(Word2Vec, GloVe)은 대용량 텍스트의 단어들의 동시 등장 통계로부터 훈련시키는 방법.



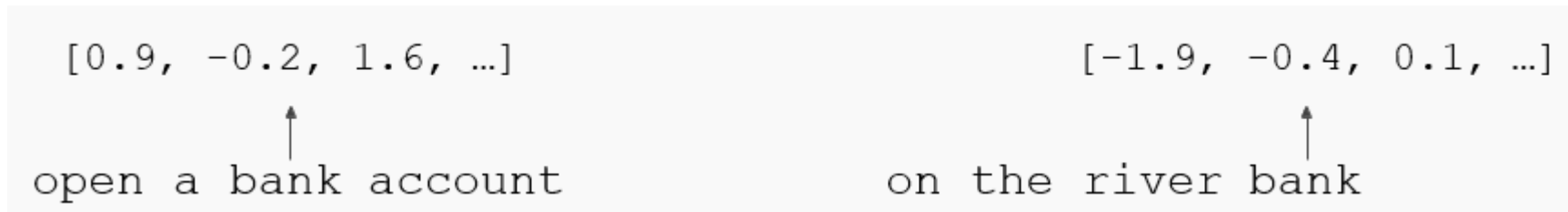
Contextual Representation

- 문제점 : 단어는 문맥에 따라서 뜻이 달라질 수 있음.

Ex) 동음이의어. 배, 사과 등.



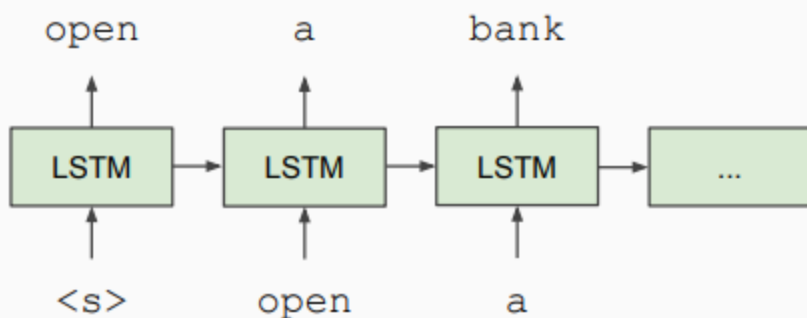
- 해결 방법 : 언어 모델을 사전 훈련 시켜서 문맥에 따른 표현(Contextual Representation)을 얻는다.



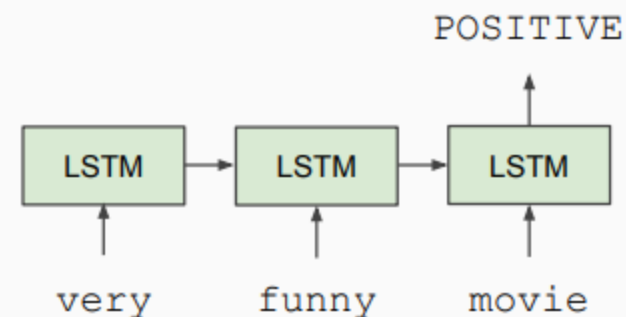
History of Contextual Representations

- *Semi-Supervised Sequence Learning, Google, 2015*

Train LSTM Language Model



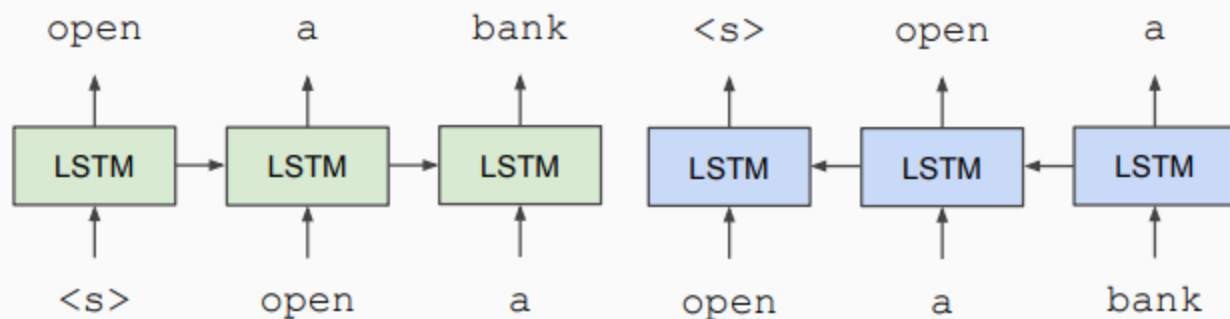
Fine-tune on Classification Task



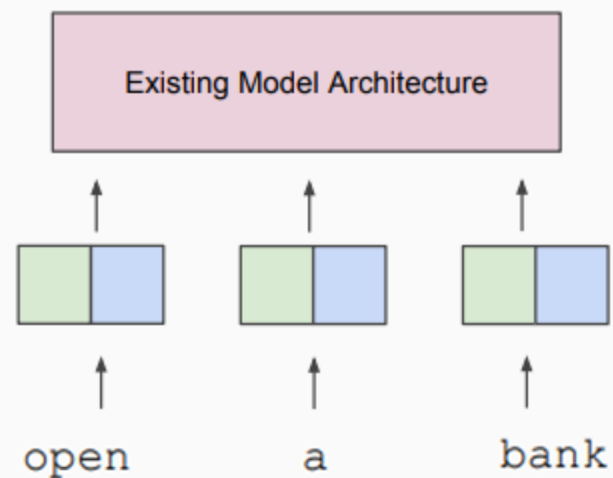
History of Contextual Representations

- *ELMo: Deep Contextual Word Embeddings*, AI2 & University of Washington, 2017

Train Separate Left-to-Right and Right-to-Left LMs



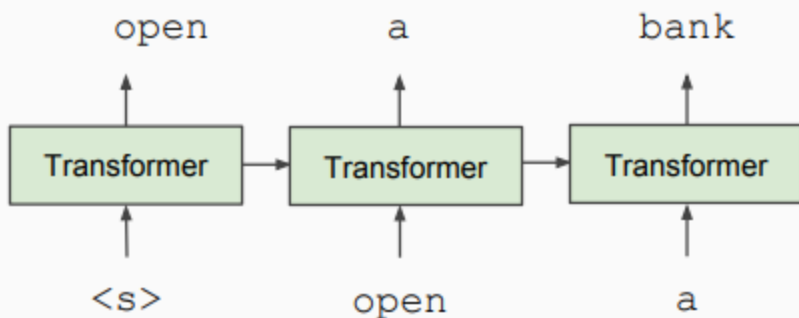
Apply as “Pre-trained Embeddings”



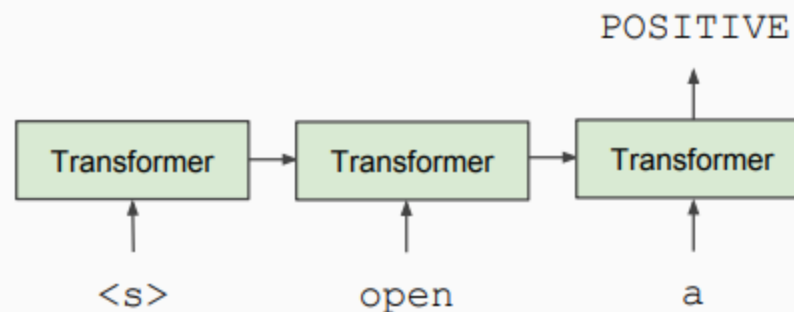
History of Contextual Representations

- *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018

**Train Deep (12-layer)
Transformer LM**



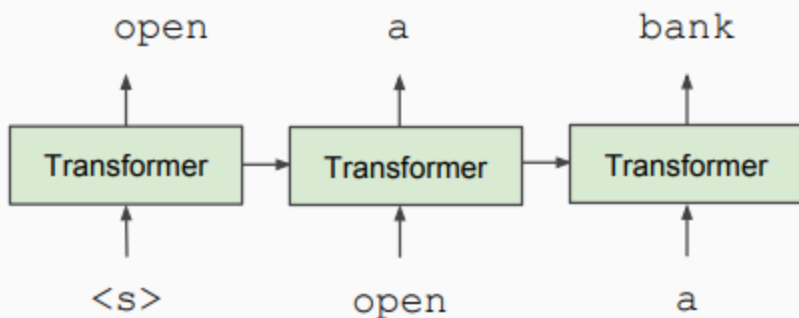
**Fine-tune on
Classification Task**



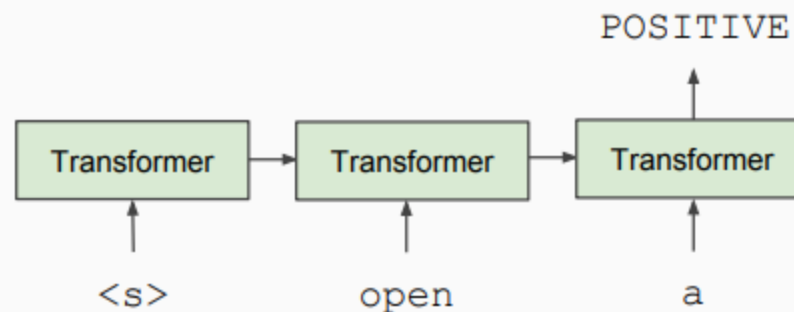
History of Contextual Representations

- *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018 이 모델을 GPT라고 한다.

**Train Deep (12-layer)
Transformer LM**



**Fine-tune on
Classification Task**



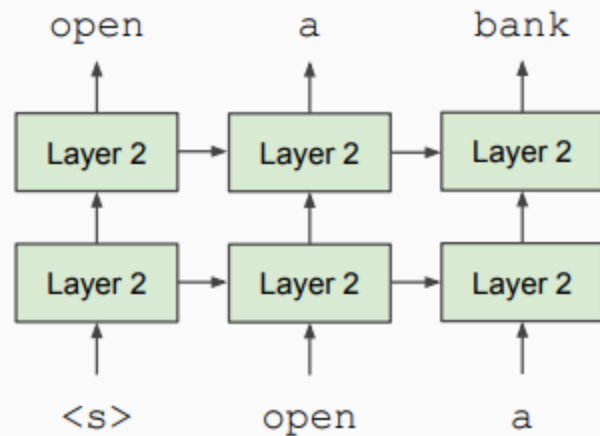
Problem with Previous Methods

- 문제점 : 언어 모델은 정방향 또는 역방향으로만 진행된다.
하지만 사실 언어의 문맥이라는 것은 양방향을 통해서 결정이 된다.
- 그렇다면 왜 언어 모델은 단방향(Unidirectional)이어야만 할까?
- 이유 : 양방향 언어모델을 만들 경우,
미리 예측할 단어에 대한 정보를 보게되기 때문.

Problem with Previous Methods

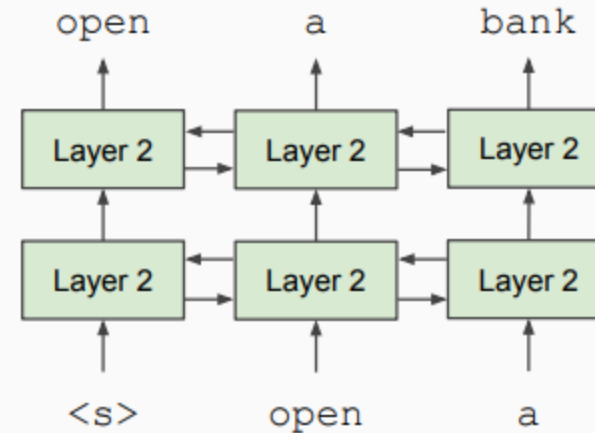
Unidirectional context

Build representation incrementally



Bidirectional context

Words can “see themselves”



Masked Language Model

- **해결 방법**: 훈련 데이터의 단어 중 $k\%$ 의 단어를 [MASK]로 변경하여 이를 예측하도록 한다.

○ We always use $k = 15\%$

store gallon
↑ ↑
the man went to the [MASK] to buy a [MASK] of milk

Masked Language Model을 제안한 논문에서는 $k = 15$ 로 사용.

Masked Language Model

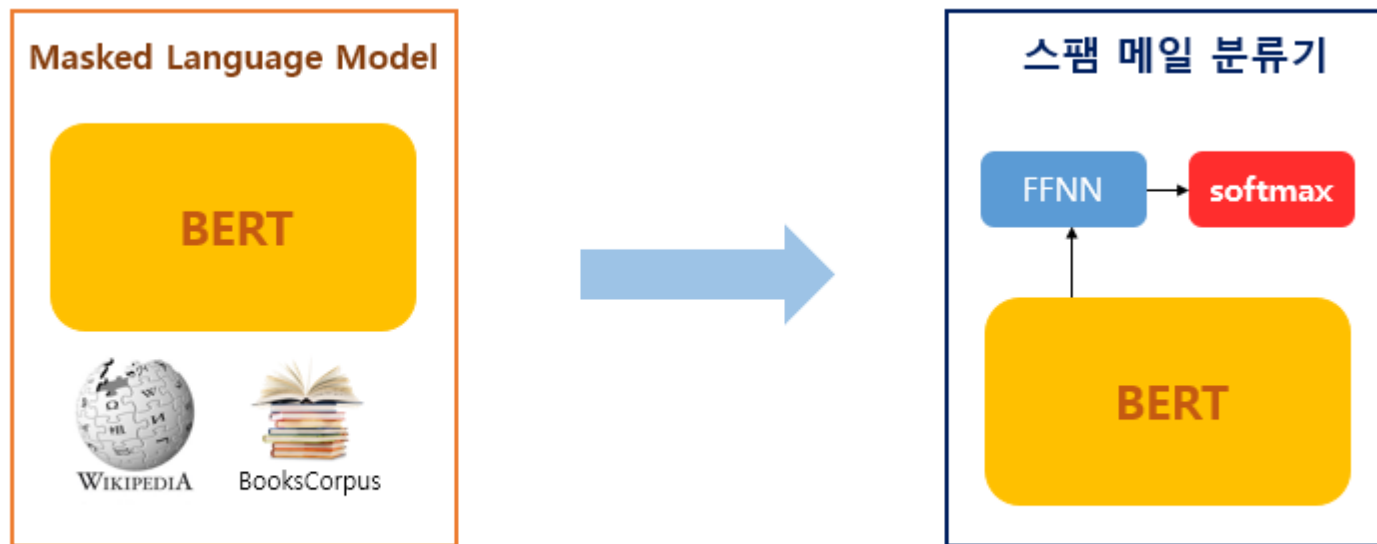
15%의 [MASK] token을 만들어 낼 때 몇 가지 추가적인 처리를 해준다.

- 80%의 경우 : token을 [MASK]로 바꾼다.
eg., my dog is hairy -> my dog is [MASK]
- 10%의 경우 : token을 random word로 바꾼다.
eg., my dog is hariy -> my dog is apple
- 10%의 경우 : token을 원래의 단어로 그대로 놔둔다.
이는 실제 관측된 단어에 대한 정보 또한 남겨주기 위해서이다.

BERT

BERT의 적용

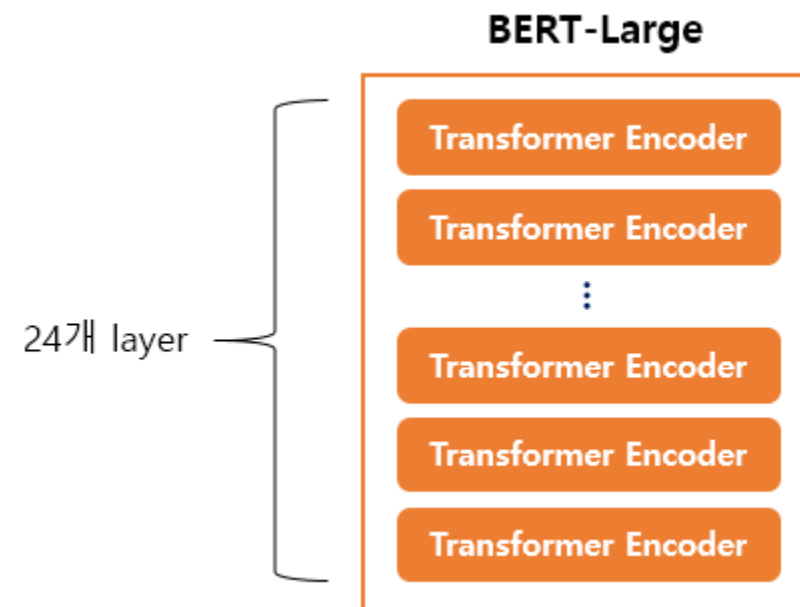
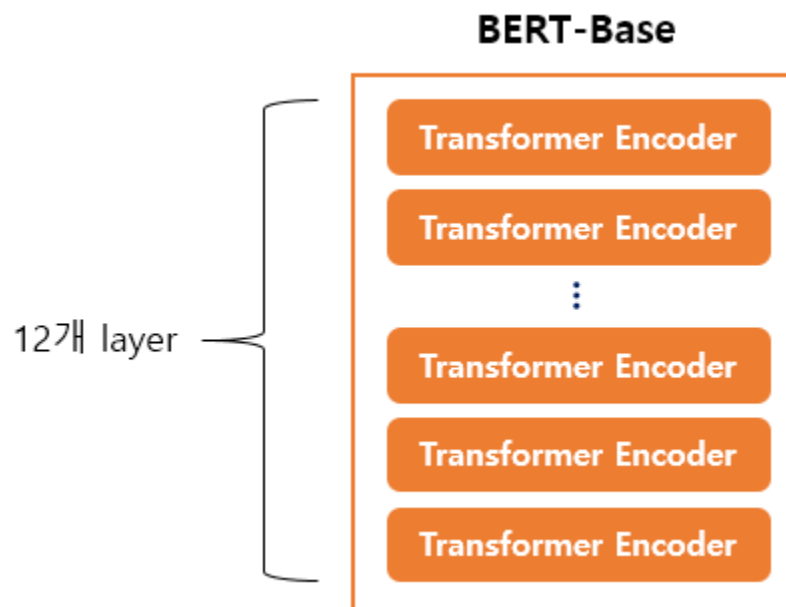
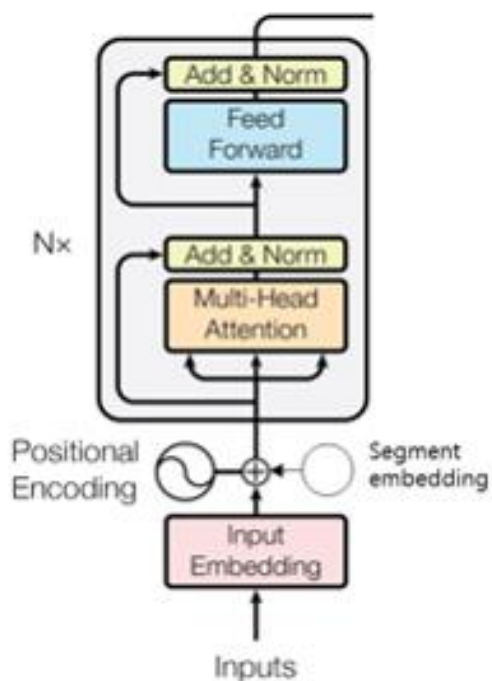
- 방대한 데이터로 사전 훈련된 언어 모델 BERT.
- BERT에 풀고자 하는 태크스에 맞는 추가적인 신경망을 추가.
- 풀고자 하는 태스크의 오차에 맞추어서 BERT를 학습. 이를 파인 튜닝(fine-tuning)이라 한다.



33억 단어에 대해서 4일간 학습시킨 언어 모델

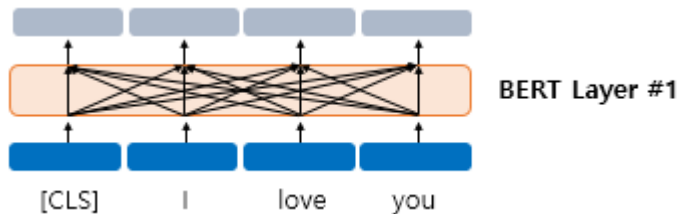
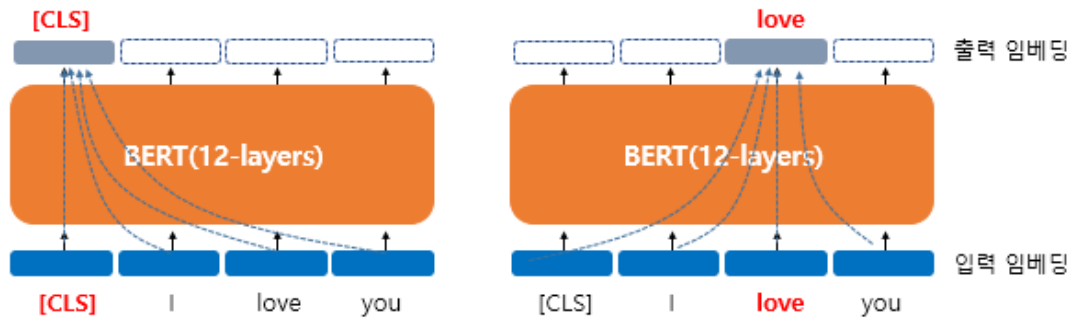
BERT-Base Vs. BERT-Large

- BERT-BASE는 트랜스포머의 인코더를 12층을 적재.
- BERT-LARGE는 트랜스포머의 인코더를 24층을 적재.



BERT의 문맥을 반영한 임베딩

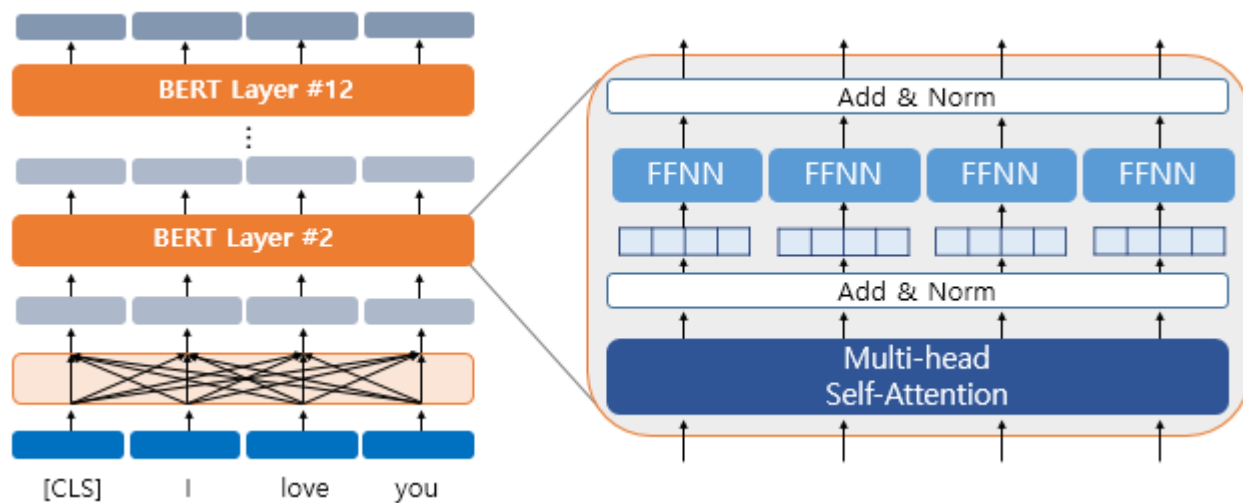
- BERT는 ELMo나 GPT-1과 마찬가지로 문맥을 반영한 임베딩(Contextual Embedding).
- BERT의 입력 임베딩의 차원은 768, 출력 또한 768.
- BERT의 출력 임베딩은 문맥을 반영한 임베딩.



셀프 어텐션을 통해 다른 단어들을 모두 참고.

BERT의 문맥을 반영한 임베딩

- BERT는 다수의 층으로 구성되어 있으며 1층의 출력은 2층의 입력으로 사용된다.



기존의 모델들

BERT 이전의 Pre-trained 모델은 크게 두 가지로 구성된다.

- Feature-based (사전 훈련된 모델의 파라미터를 고정)
 - **ELMo**
 - **Shallowly** 양방향 언어 모델(biLM)
 - **ELMo의 경우 양방향이지만 진정한 양방향이 아니다.**
 - 언어 모델의 파라미터는 Freezing하고,
각 layer에 대한 비율 조정 파라미터(γ)를 계산
 - $[\gamma_{input}, \gamma_{layer1}, \gamma_{layer2}, \gamma_{layer3}]$
- Fine-tuning (사전 훈련된 모델의 파라미터를 함께 학습)
 - **GPT**
 - **단방향(Unidirectional)** 언어 모델(language models)
 - **GPT의 경우 단방향 모델이다.**

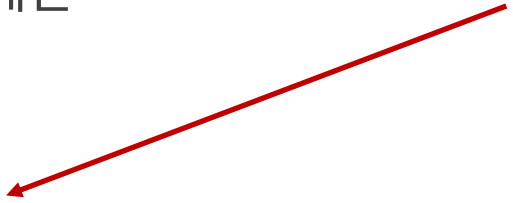
기존의 모델들

BERT 이전의 Pre-trained 모델은 크게 두 가지로 구성된다.

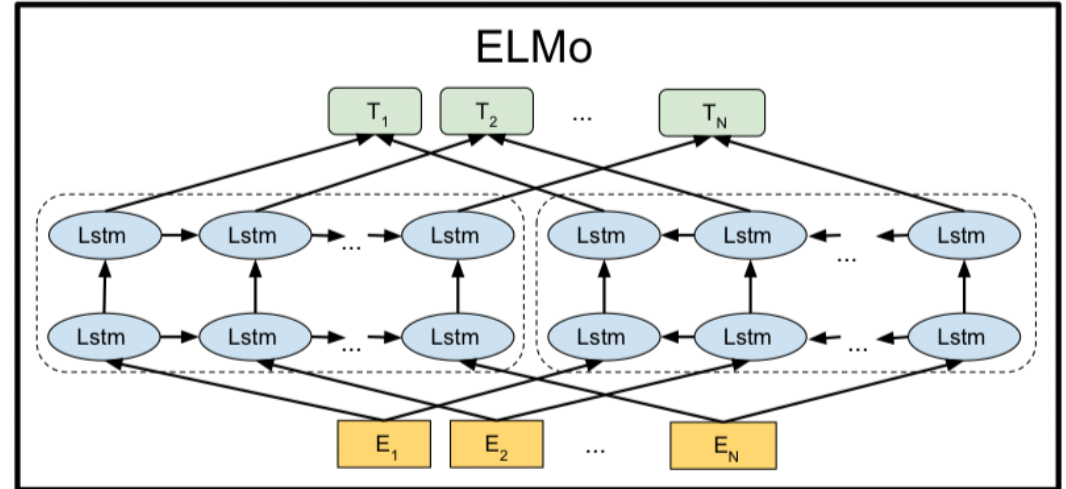
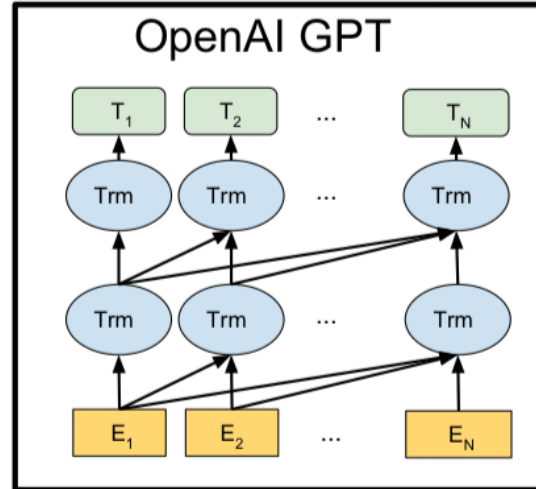
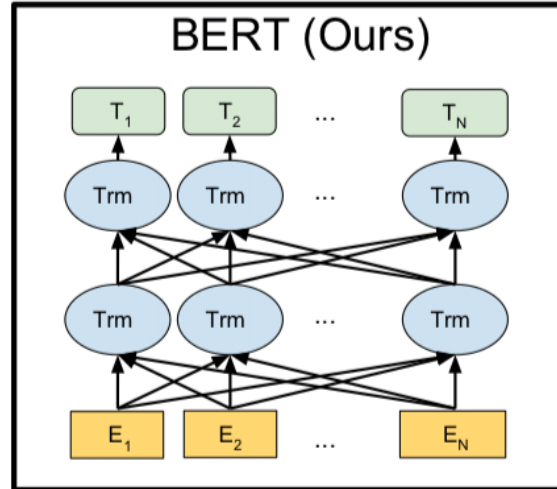
- Feature-based (사전 훈련된 모델의 파라미터를 고정)
 - **ELMo**
 - **Shallowly** 양방향 언어 모델(biLM)
 - **ELMo의 경우 양방향이지만 진정한 양방향이 아니다.**
 - 언어 모델의 파라미터는 Freezing하고,
각 layer에 대한 비율 조정 파라미터(γ)를 계산
 - $[\gamma_{input}, \gamma_{layer1}, \gamma_{layer2}, \gamma_{layer3}]$
- Fine-tuning (사전 훈련된 모델의 파라미터를 함께 학습)
 - **GPT**
 - **단방향(Unidirectional)** 언어 모델(language models)
 - **GPT의 경우 단방향 모델이다.**

BERT

Fine-tuning 방식을 취하되,
GPT와는 달리 양방향을 실현한다.



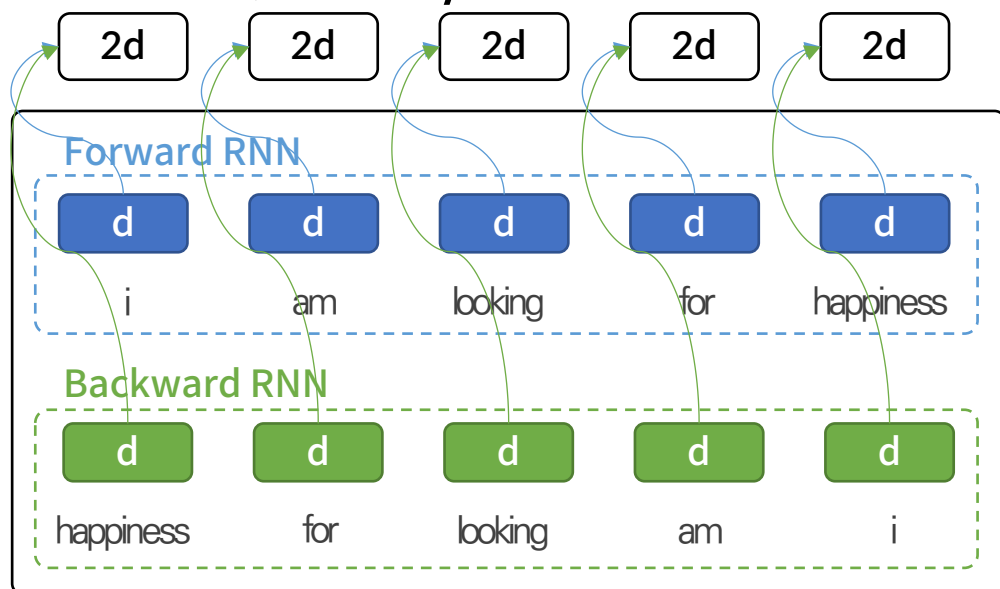
기존의 모델들



Un-directional Vs. Bi-directional

- RNN Language Model, Transformer: 이전 단어들을 보고 다음 단어를 예측(**Uni-directional**)
- ELMo => BiLM : Shallowly Bi-directional
- BERT
 - Masked Language Model : 문장에 Masking 15% 처리한 후 Mask된 단어를 예측
 - 1. 주변 단어들을 보고 Masked 단어를 예측 (**Bi-directional**)

Bidirectional RNN(shallowly 양방향)



Bidirectional BERT(deeply 양방향)

i am looking for happiness . I am Donghwa

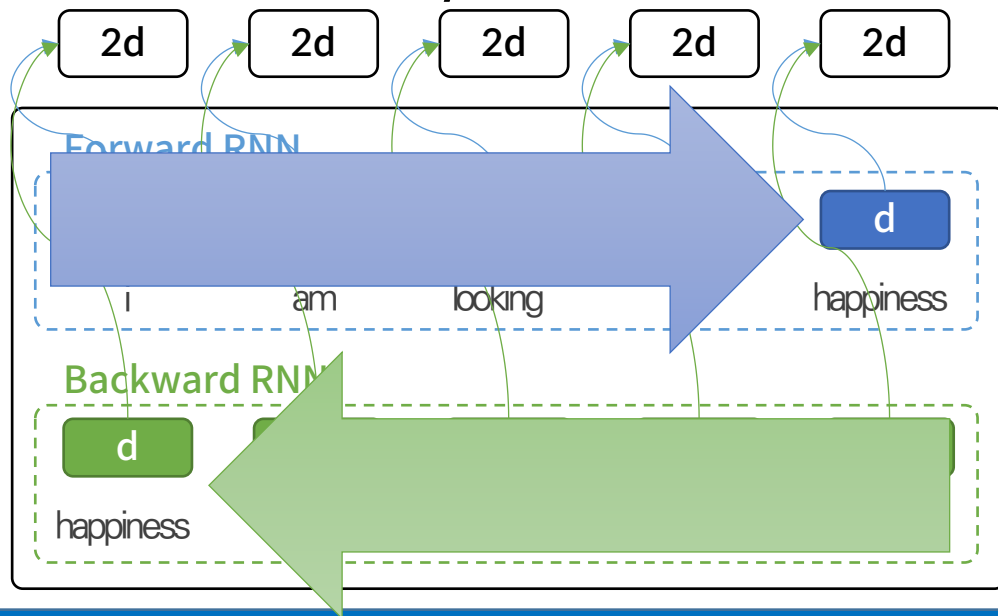
Input: i am looking for [MASK1] . I am [MASK2]

Labels: [MASK1] = happiness; [MASK2] = Donghwa

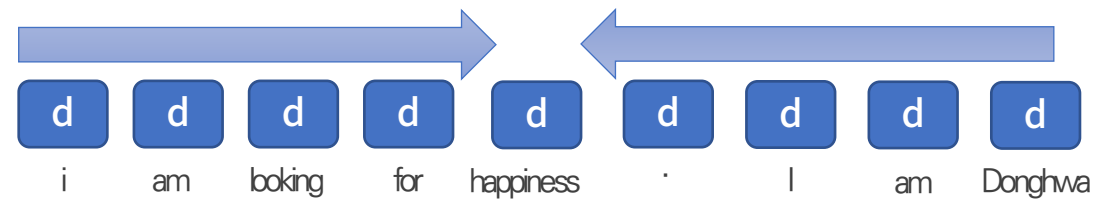
Un-directional Vs. Bi-directional

- RNN Language Model, Transformer: 이전 단어들을 보고 다음 단어를 예측(**Uni-directional**)
- ELMo => BiLM : Shallowly Bi-directional
- BERT
 - Masked Language Model : 문장에 Masking 15% 처리한 후 Mask된 단어를 예측
 - 1. 주변 단어들을 보고 Masked 단어를 예측 (**Bi-directional**)

Bidirectional RNN(shallowly)



Bidirectional BERT(deeply)



Input: i am looking for [MASK1] . I am [MASK2]

Labels: [MASK1] = happiness; [MASK2] = Donghwa

Next Sentence Prediction

- BERT
 - 2. 문장들 사이의 관계를 학습하기 위해 “다음 문장 예측 태스크를 도입

BERT에서는 훈련 데이터에서 두 문장을 이어붙여 원래의 훈련 데이터에서 붙여져 있던 문장인지를 맞추는 다음 문장 예측 문제를 수행한다.

50% : Sentence A, B가 실제 다음 문장

50% : Sentence A, B가 랜덤으로 뽑힌 관계가 없는 두 문장

Sentence A: i am looking for happiness

Sentence B: i am Wonjoon

Labels: **Is**NextSentence

Sentence A: i am looking for happiness

Sentence B: i am Sana

Labels: **Not**NextSentence

BERT의 Pre-training

결론적으로 BERT는 사전 학습 단계에서 두 가지 방법으로 학습된다.

- 1. **Masked Language Model: 문장에 Masking 15% 처리한 후 Mask된 단어를 예측**
- 2. 문장들 사이의 관계를 학습하기 위해 “다음 문장 예측 태스크를 도입”

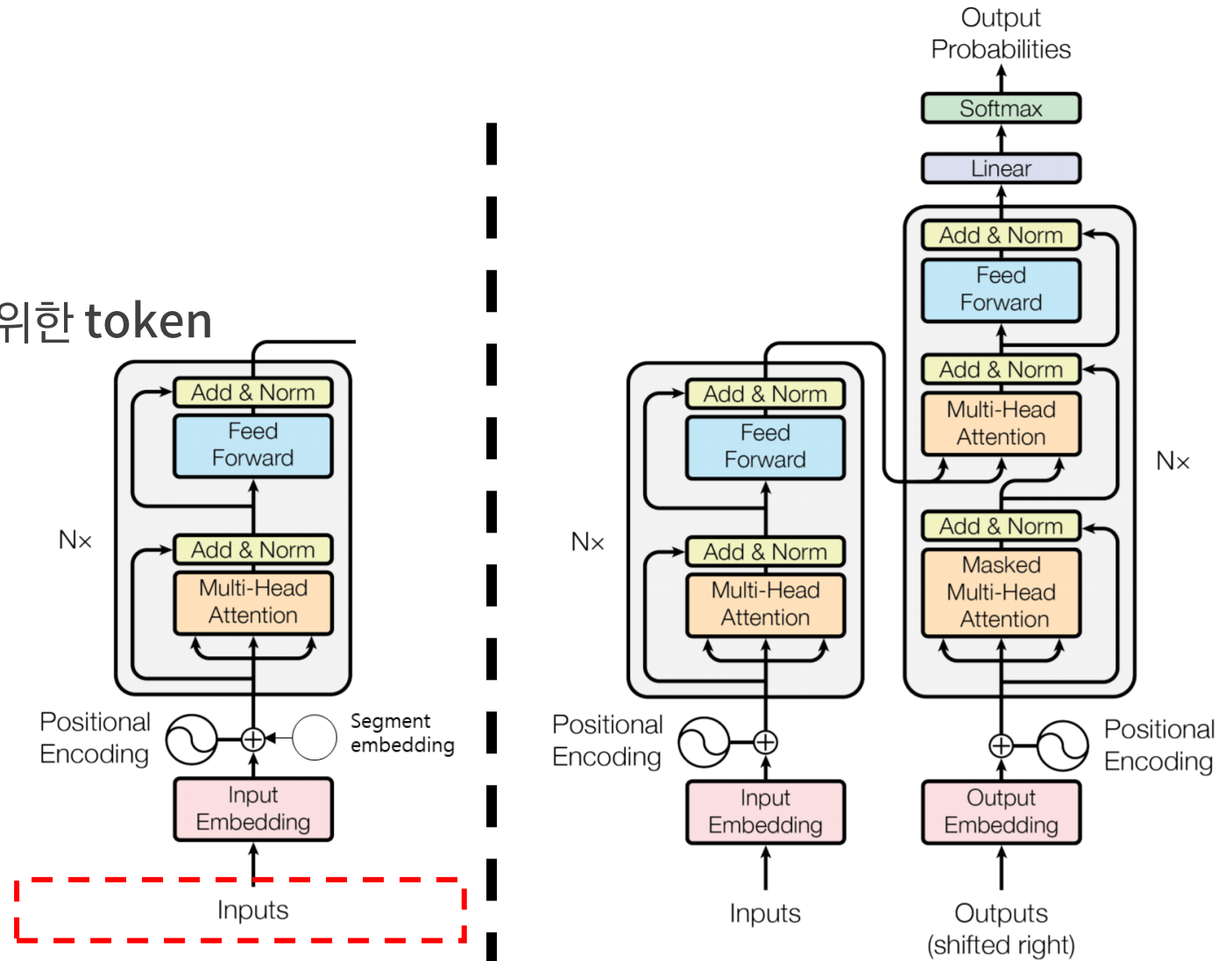
사실 BERT의 성능의 대부분은 1. Masked Language Model에서 나오는 편이고,
2.의 경우에는 BERT가 향후 파인 튜닝 시에 두 개의 문장을 입력으로 받는 태스크를 풀 경우를 위해 추가한 것.

Ex) 두 개의 문장을 입력받고 이 문장이 모순 관계인지, 중립 관계인지를 맞추는 태스크 등.

실제로 BERT 이후의 연구들에서는 2.를 그다지 중요하게 생각하지 않는 경우가 많음.

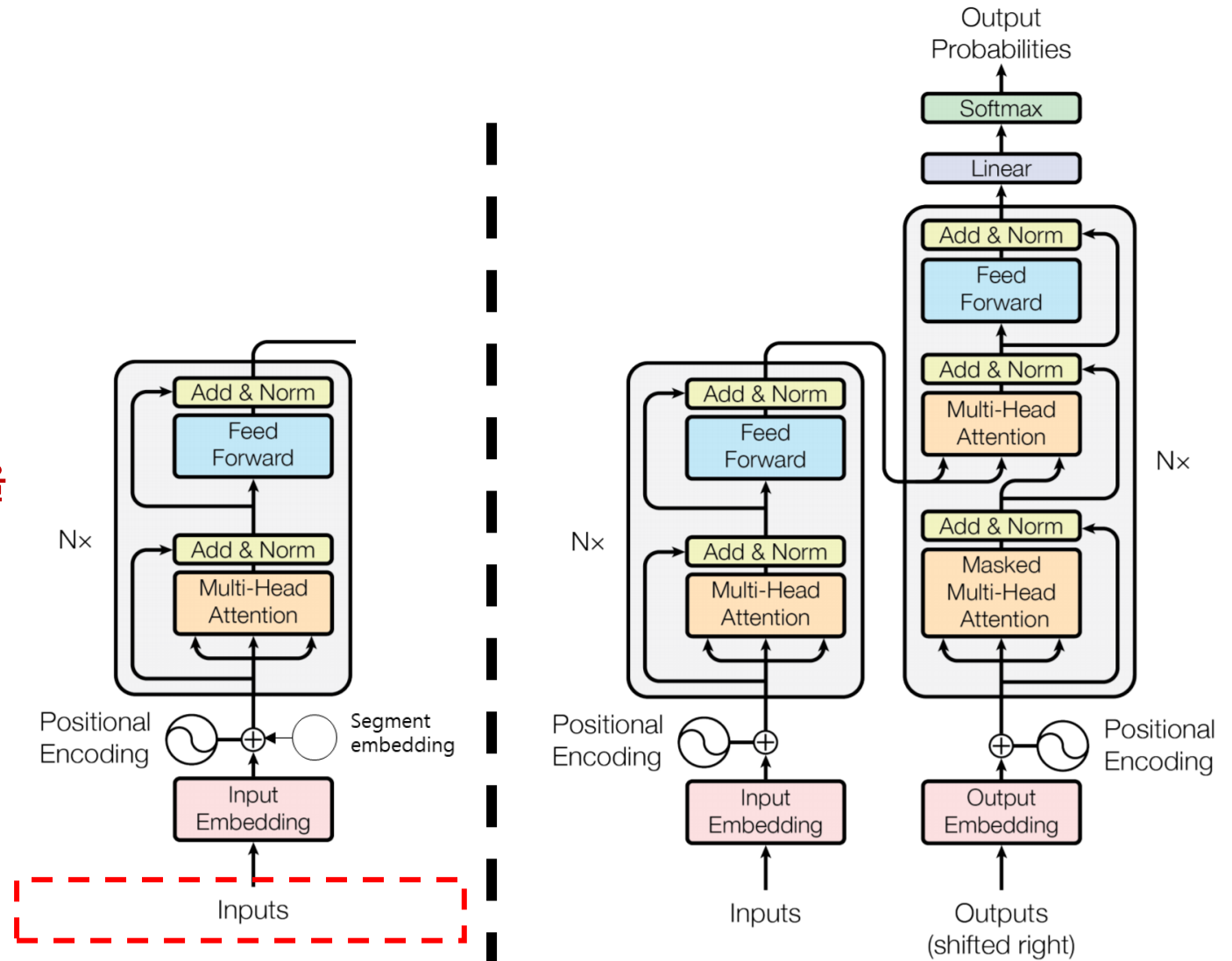
Transformer Vs. BERT

- Transformer
 - [SOS]: 문장의 시작을 알리는 token
 - [EOS]: 문장의 끝을 알리는 token
- BERT
 - [CLS]: Task-specific한 정보를 주기 위한 token
 - [SEP]: 문장 A, B를 구분하는 token
 - [MASK]: 예측하는 Target



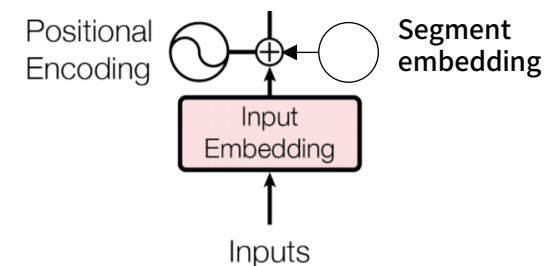
Transformer Vs. BERT

- Transformer
 - Input embedding
 - Positional embedding
- BERT
 - Input embedding
 - Positional embedding
 - **Segment embedding 추가**
 - **Token A group, Token B group을 구분하는 정보**



BERT의 세 가지 Embedding

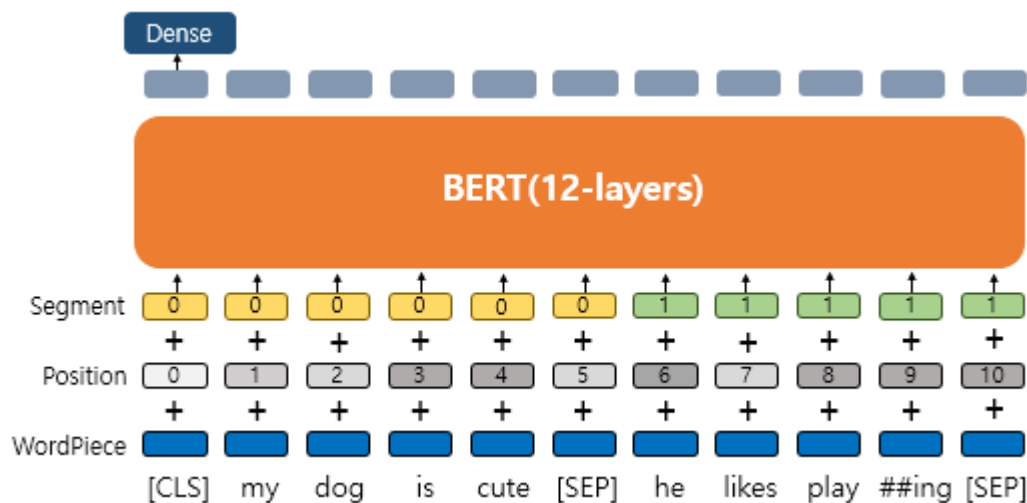
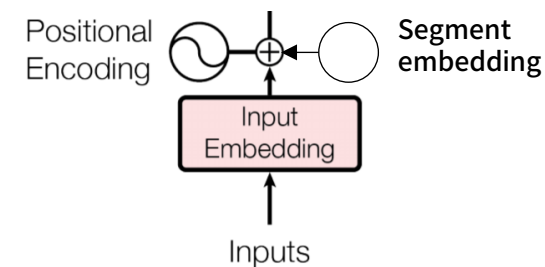
- BERT에는 총 세 가지 Embedding이 사용된다.
- Token Embedding은 우리가 알고있는 Input Word Embedding
- Segment Embedding은 두 개의 문장을 구분하기 위한 Embedding
- Position Embedding은 트랜스포머의 Positional Encoding을 대체할 수 있는 방법



Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

BERT의 세 가지 Embedding

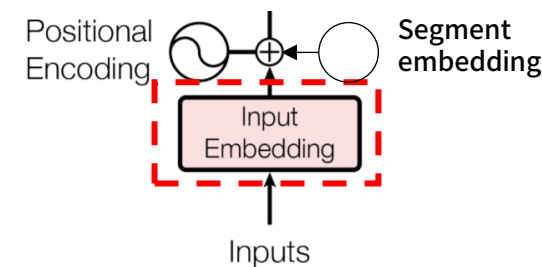
- BERT에는 총 세 가지 Embedding이 사용된다.
- Token Embedding은 우리가 알고있는 Input Word Embedding
- Segment Embedding은 두 개의 문장을 구분하기 위한 Embedding
- Position Embedding은 트랜스포머의 Positional Encoding을 대체할 수 있는 방법



Token Embedding == Word Embedding

- Encoder

- Sent1: [CLS] i am looking for happiness [SEP], B type sentence [SEP]



looking x_j

Vocab size(Encoder)

$\langle \text{PAD} \rangle$	$\langle s \rangle$	$\langle /s \rangle$	$\langle \text{UNK} \rangle$	am	for	...	i	looking	...	w	...	z
0	0	0	0	0	0	0	0	1	0	0	0	0

Embed size

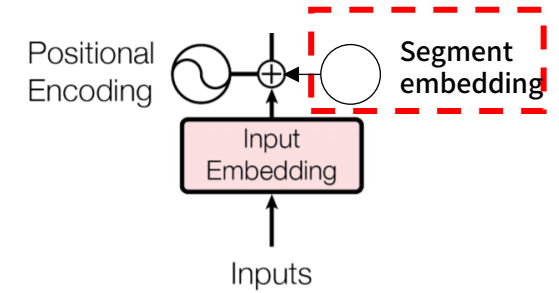
$\langle \text{PAD} \rangle$	0.0	0.0	0.0	0.0
$\vdots$	0.0	0.3	0.7	0.0
looking	0.2	0.1	0.7	0.2
$\vdots$	0.9	0.1	0.7	0.9
$\vdots$	0.5	0.7	0.1	0.5
$\vdots$	0.2	0.9	0.4	0.2
z	0.2	0.0	0.7	0.2

0.2	0.1	0.7	0.2
-----	-----	-----	-----

w_j looking

Segment Embedding

- Encoder
 - Sent1: [CLS] i am looking for happiness [SEP], B type sentence [SEP]



looking x_j

Vocab size(Encoder)

A	B
1	0

Embed size

A				
B				

0.1	0.1	0.1	0.3
-----	-----	-----	-----

w_j looking

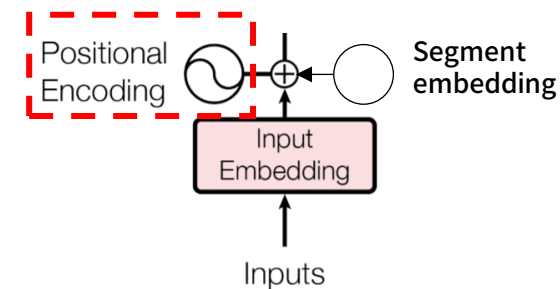
Position Embedding

- Encoder
 - Sent1: [CLS] i am looking for happiness [SEP], B type sentence [SEP]

3th position

Position vocab					
0	...	3	...	9	10
0	...	1	0	0	0

Embed size				
0	0.1	0.4	0.7	0.1
⋮	0.0	0.3	0.7	0.0
3	0.1	0.1	0.9	0.1
⋮	...	...	...	...
10	0.2	0.0	0.7	0.2



3th position embedding

0.1	0.1	0.9	0.1
-----	-----	-----	-----

Position Embedding

- Encoder

- Sent1: [CLS] i am looking for happiness [SEP], B type sentence [SEP]

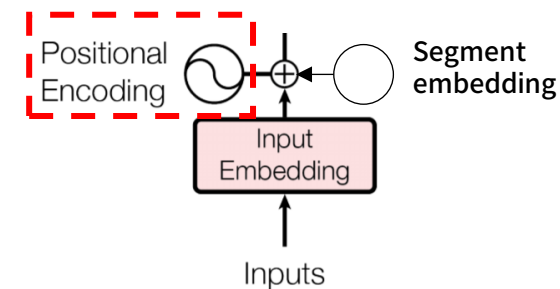
0 1 2 3 4 5 6 7 8 9 10

샘플의 길이가 11이라고 가정해보자.

3th position

Position vocab					
0	...	3	...	9	10
0	...	1	0	0	0

Embed size				
0	0.1	0.4	0.7	0.1
⋮	0.0	0.3	0.7	0.0
3	0.1	0.1	0.9	0.1
⋮	...	...	...	...
10	0.2	0.0	0.7	0.2

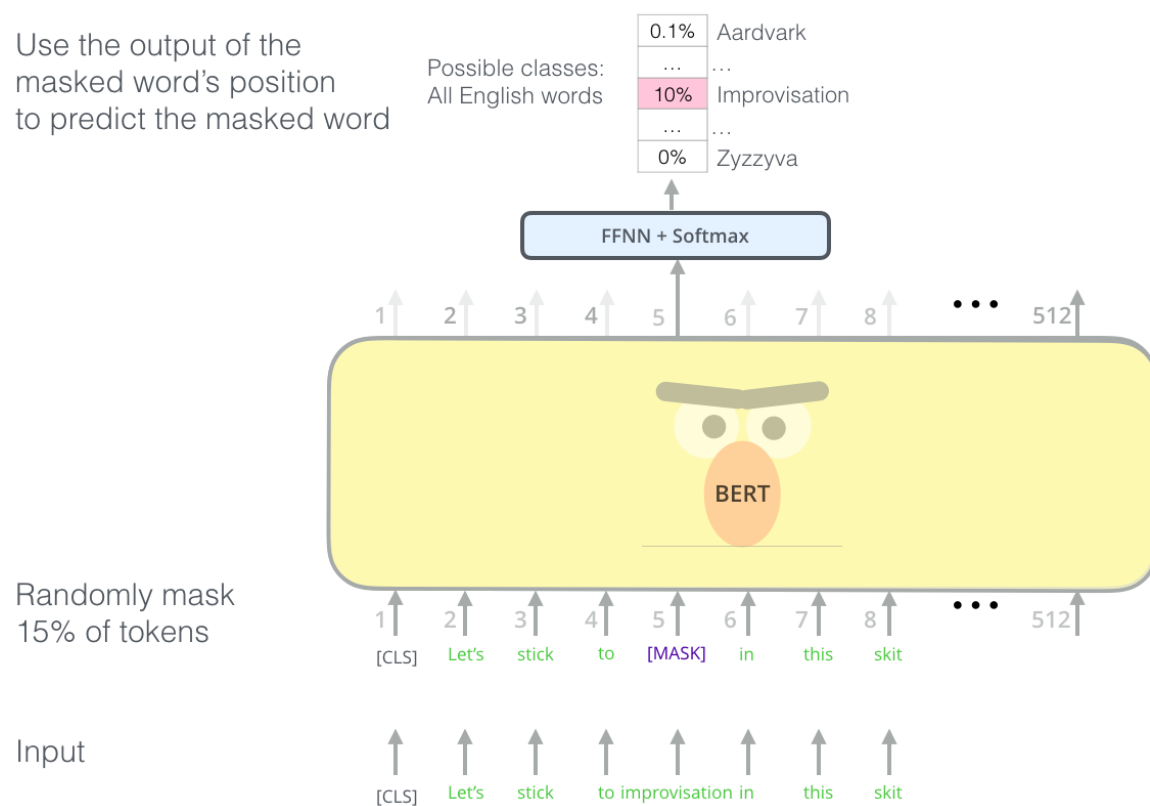


3th position embedding

0.1 0.1 0.9 0.1

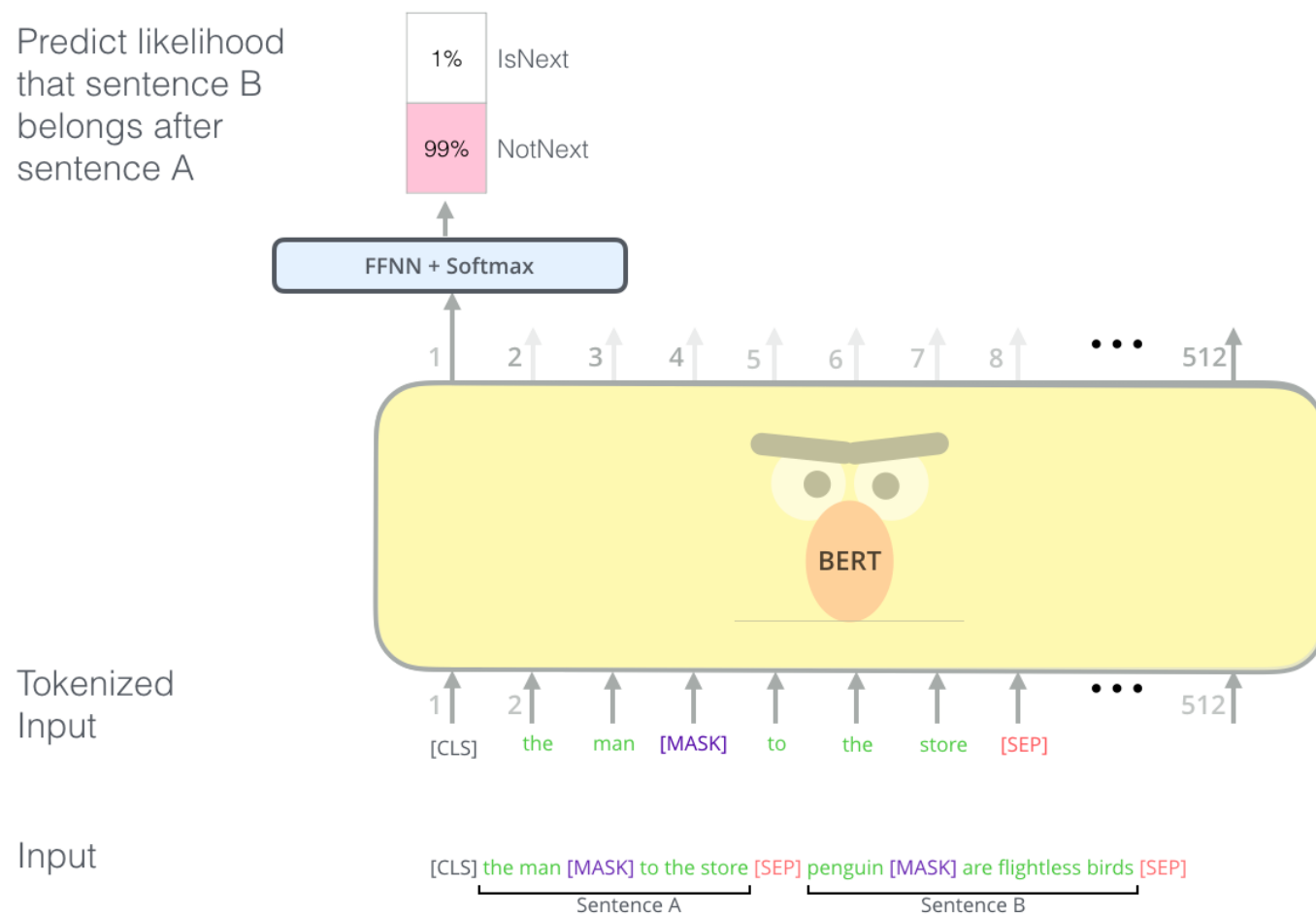
BERT의 사전 훈련

- BERT는 사전 훈련 시 첫번째 Task로 Masked Language Model을 학습



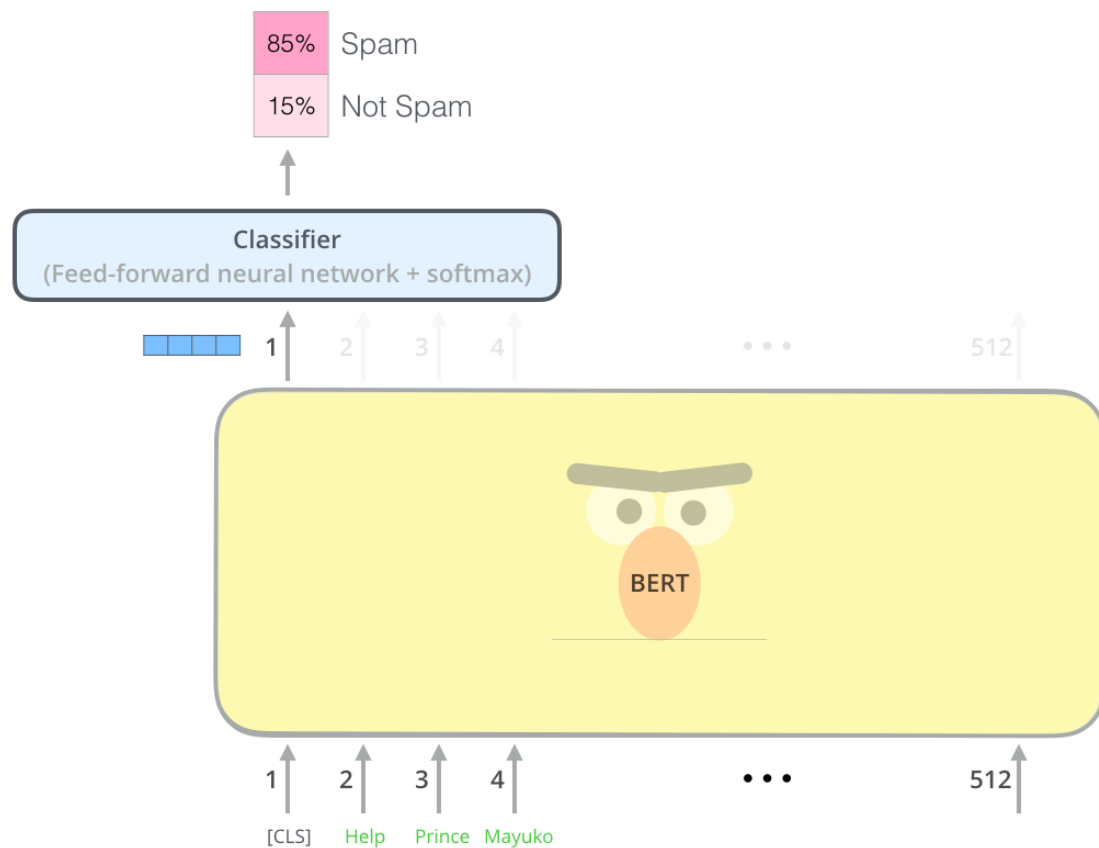
BERT의 사전 훈련

- BERT는 사전 훈련 시 두번째 Task로 다음 문장 예측



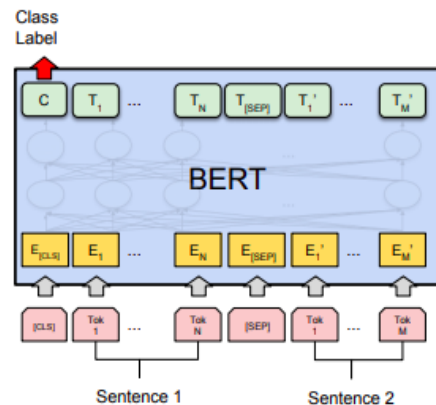
BERT의 Fine-tuning

- BERT의 윗단의 신경망을 통해서 원하는 Task를 수행.
- 텍스트 분류의 경우 <CLS>의 토큰을 최종 Classification을 위한 입력으로 사용.

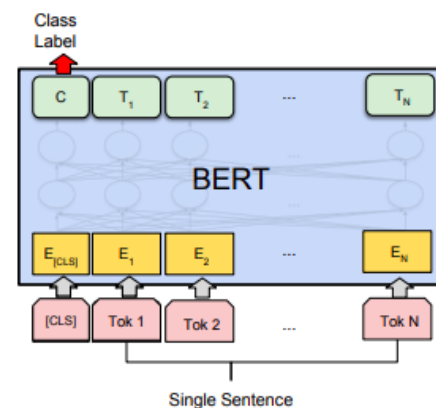


BERT의 다양한 적용

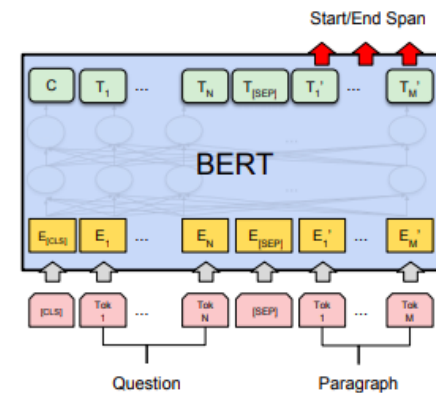
- BERT를 다양한 태스크에 적용 가능.
- Text Classification
- Named Entity Recognition
- QA
- NLI



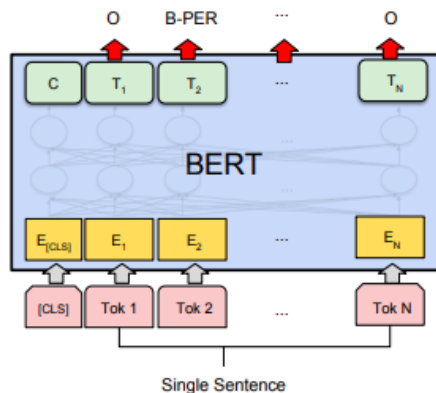
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



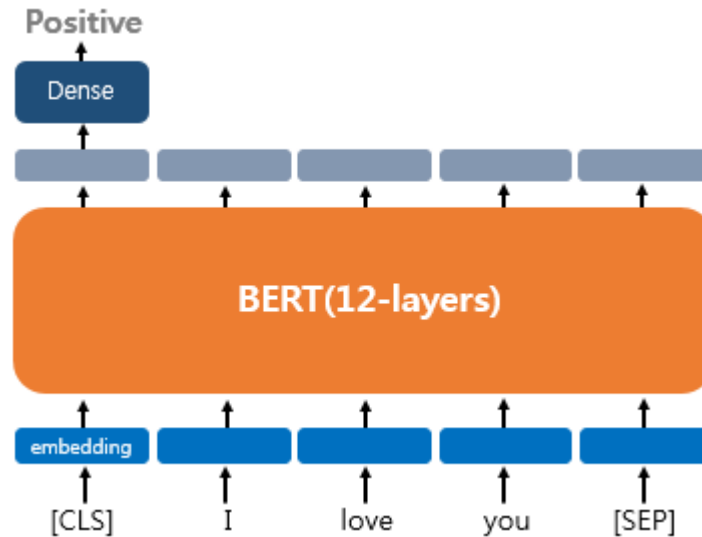
(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

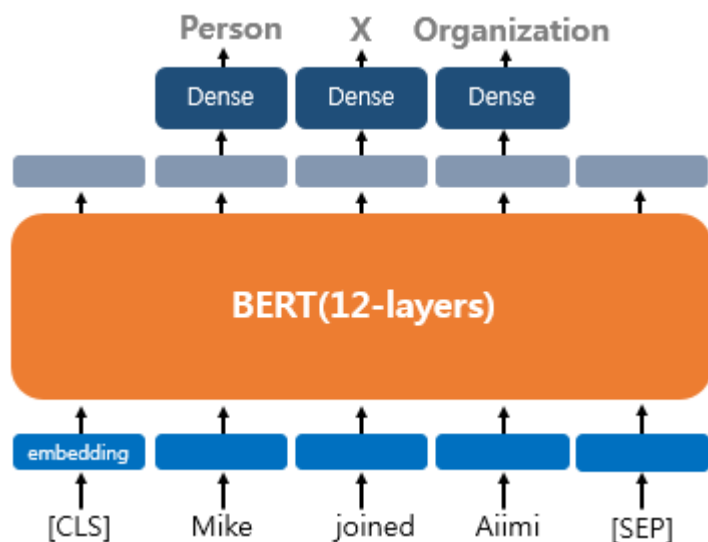
Single Text Classification

- BERT의 윗단의 신경망을 통해서 원하는 Task를 수행.
- 텍스트 분류의 경우 <CLS>의 토큰을 최종 Classification을 위한 입력으로 사용.



하나의 텍스트에 대한 태깅 작업(Tagging)

- 출력층에서는 입력 텍스트의 각 토큰의 위치에 밀집층을 사용하여 분류에 대한 예측.



retrieval : 쿼리로부터 연관된 문서를 추출하는 것

데이터 셋

[CLS] 쿼리 [SEP] 연관있는 문장 [SEP] ==> LABEL 1

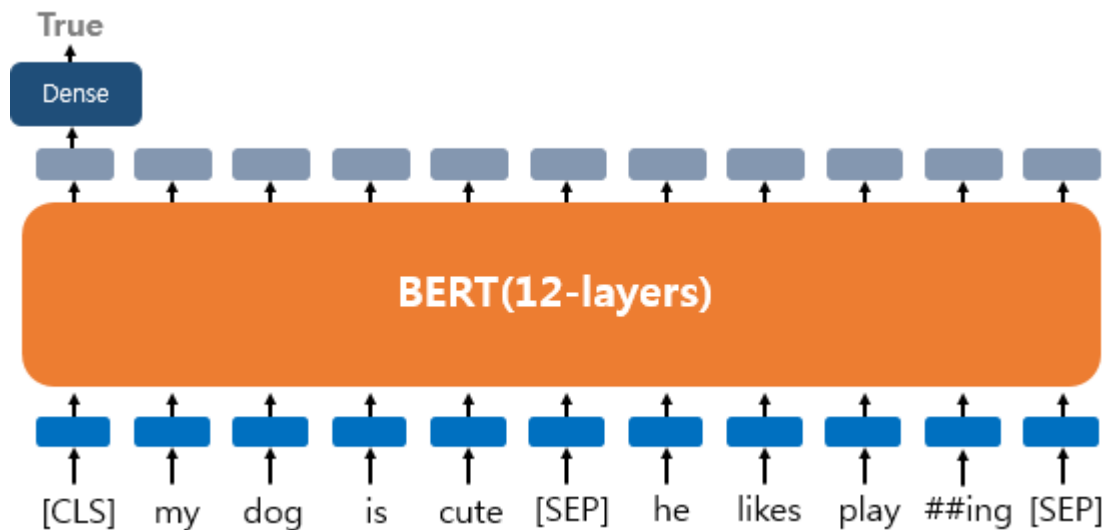
[CLS] 쿼리 [SEP] 연관없는 문장 [SEP] ==> LABEL 0

BERT

쿼리와 존재하는 모든 문서의 쌍에 대해 해당 TASK 수행

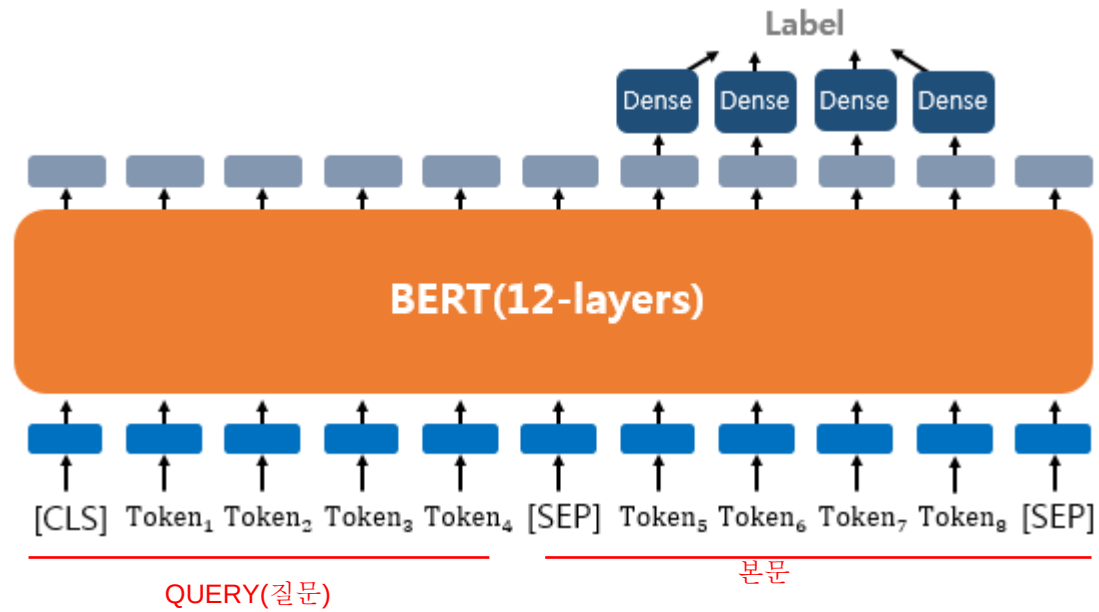
텍스트의 쌍에 대한 분류 또는 회귀 문제(Text Pair Classification or Regression)

- 텍스트의 쌍을 입력받는 이러한 태스크의 경우에는 입력 텍스트가 1개가 아니므로, 텍스트 사이에 [SEP] 토큰을 집어넣고, Sentence 0 임베딩과 Sentence 1 임베딩이라는 두 종류의 세그먼트 임베딩을 모두 사용하여 문서를 구분.

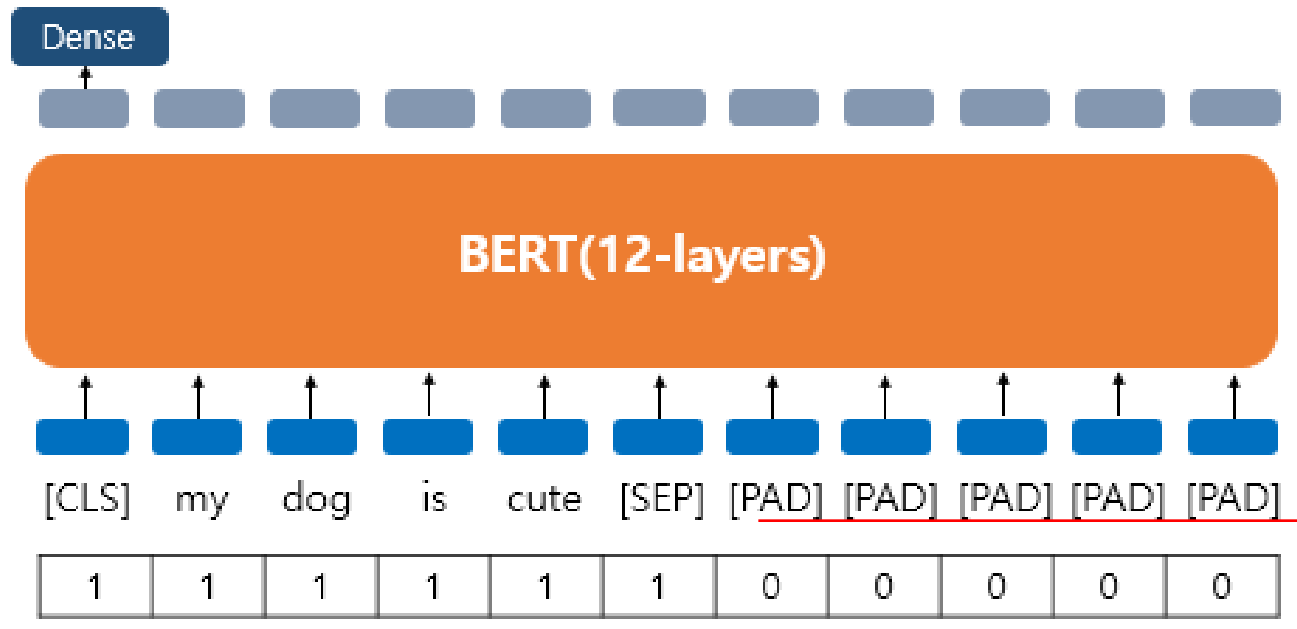


기계 독해

- 기계 독해는 질문과 본문을 입력받으면, 본문의 일부분을 추출해서 질문에 답변하는 것.



어텐션 마스크(Attention Mask)



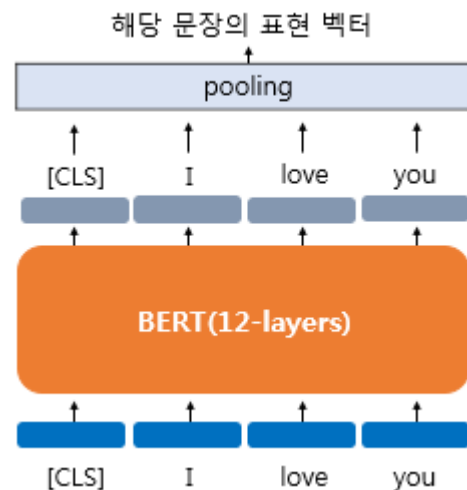
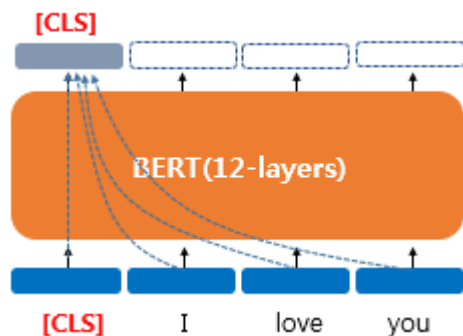
- 앞서 논문 설명에서는 언급되지 않았지만, 실제 BERT를 사용할 경우에 필요한 입력.
- 숫자 1은 해당 토큰은 실제 단어이므로 마스킹을 하지 않는다는 의미이고, 숫자 0은 해당 토큰은 패딩 토큰이므로 마스킹을 한다는 의미. 위의 그림과 같이 실제 단어의 위치에는 1, 패딩 토큰의 위치에는 0의 값을 가지는 시퀀스를 만들어 BERT의 또 다른 입력으로 사용하면 된다.

SBERT

BERT를 사용한 문장 임베딩

- BERT를 이용하여 문장 임베딩을 얻는 방법은 크게 세 가지로 나뉜다.

1. BERT의 [CLS] 토큰의 출력 벡터를 문장 벡터로 간주
2. BERT의 모든 단어의 출력 벡터에 대해서 평균 풀링을 수행한 벡터를 문장 벡터로 간주
3. BERT의 모든 단어의 출력 벡터에 대해서 맥스 풀링을 수행한 벡터를 문장 벡터로 간주



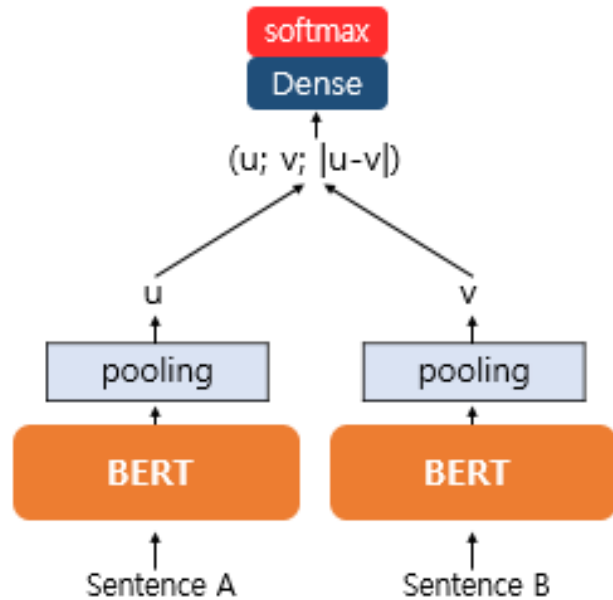
NLI 태스크

- NLI는 두 개의 문장이 주어지면 수반(entailment) 관계인지, 모순(contradiction) 관계인지, 중립(neutral) 관계인지를 맞추는 문제입니다.

문장 A	문장 B	레이블
A lady sits on a bench that is against a shopping mall.	A person sits on the seat.	Entailment
A lady sits on a bench that is against a shopping mall.	A woman is sitting against a building.	Entailment
A lady sits on a bench that is against a shopping mall.	Nobody is sitting on the bench.	Contradiction
Two women are embracing while holding to go packages.	The sisters are hugging goodbye while holding to go packages after just eating lunch.	Neutral

NLI 태스크를 이용하여 BERT의 임베딩 능력을 Up

- NLI 태스크를 푸는 과정에서 BERT는 문장 임베딩 능력이 상승하게 됨.
- 이후 테스트 과정에서는 BERT의 Pooling을 문장 임베딩으로 간주.



$$h = (u; v; |u - v|)$$

$$o = \text{softmax}(W_y h)$$

BERTSum

BERTSum

- Original BERT와 달리 모든 문장의 앞에 [CLS] 토큰을 추가.
- Segment Embedding을 번갈아서 사용.
- 각 i번째 문장에 대해서 요약문에 포함시킬 것인지 이진 분류.
- 이때 출력층 이전에 단순히 MLP, LSTM, Trm Layer 등을 사용.

